

STM32F446RE

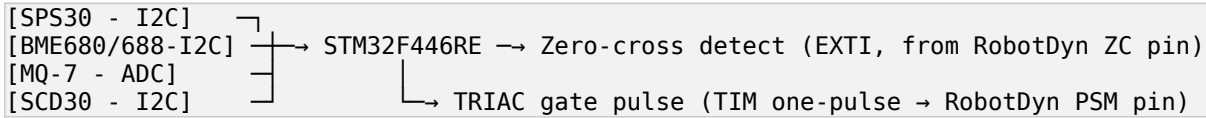
Air-Quality-Driven AC Motor (Fan) Controller

System Design & STM32CubeMX Configuration Reference

Sensors: SPS30 • BME680/688 • MQ-7 • SCD30 | Actuator: RobotDyn AC Dimmer (TRIAC) | HAL Driver Layer

1. System Overview

This document summarizes the design for an air-quality-driven ventilation controller built on the STM32 Nucleo-F446RE. The system reads ambient air-quality data from four sensors, computes a ventilation demand, and drives an AC extractor/ventilation fan via phase-angle control through a RobotDyn AC dimmer module.



2. Sensor Interfaces

Sensor	Interface	Notes
SPS30	I2C (or UART)	I2C simpler, shares bus with BME680. Address 0x69. Keep bus ≤ 100 kHz (clock-stretch sensitivity).
BME680 / 688	I2C	Address 0x76 or 0x77 (via SDO pin). Real IAQ output needs Bosch BSEC library — plain HAL only gives raw gas resistance.
SCD30	I2C	Max 100 kHz. Clock-stretches up to ~ 150 ms during measurement — the bus-timing constraint. Give it a dedicated I2C peripheral.
MQ-7	Analog (ADC) + PWM heater drive	Needs heater duty-cycling (~ 60 s high-power / ~ 90 s low-power phase typical). Driven via a dedicated PWM timer channel + MOSFET rather than a plain GPIO switch, so the low-power phase can be emulated as a duty-cycle rather than a second voltage rail.

⚠ Bus conflict: SCD30's aggressive clock stretching can starve a shared I2C bus. Put SCD30 on its own peripheral (I2C1); group SPS30 + BME680 on a second peripheral (I2C2). The F446RE has 3 I2C peripherals, so this is easy to accommodate.

3. AC Dimmer — RobotDyn Module

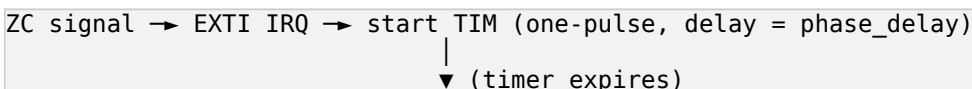
The RobotDyn AC Dimmer is a single-channel TRIAC-based phase-cut controller with the zero-cross detector, opto-isolation, and TRIAC gate driver already built onto the board. The STM32 only ever talks to two logic-level pins:

- ZC (zero-cross output) — pulses/edges once per zero-crossing, opto-isolated from mains.
- PSM (dimming / gate control input) — driven by the STM32 to fire the TRIAC at a calculated delay after each zero-cross.

Because isolation and zero-cross detection are handled on-board, no external MOC3021/3041 opto-driver is required — this simplifies the original generic-TRIAC design.

3.1 Operating Principle

At 50 Hz mains, zero-crossing occurs 100 times per second (every 10 ms); at 60 Hz, 120 times per second (every ~ 8.33 ms). On each ZC edge, the STM32 starts a delay timer; the longer the delay before firing PSM, the less power reaches the load (phase-cut dimming).



half-cycle Fire short gate pulse on PSM (~50-100 μ s) → TRIAC latches for rest of

Using delay() inside the ISR (common in Arduino examples) causes long blocking interrupts at low power levels. Using a hardware timer (TIM one-pulse mode) instead keeps the zero-cross interrupt short and non-blocking — this is the approach used in this design.

⚠ Motor caveat: a fan motor is an inductive load, unlike a resistive lamp. Simple phase-cut dimming can cause audible hum/buzz and reduced torque at low speed, and may stall the motor below a usable threshold. Test the actual fan's usable dimming range — many AC fan motors only dim smoothly from roughly 40-100%. Below that, cycle-skipping (burst-fire) control may be needed instead of phase-cut.

4. Peripheral & Pin Allocation (Nucleo-F446RE)

Function	Peripheral	Pin(s)
SCD30	I2C1	PB6 (SCL), PB7 (SDA)
SPS30 + BME680	I2C2	PB10 (SCL), PB3 (SDA)
MQ-7 analog read	ADC1, IN0	PA0
MQ-7 heater cycle (PWM, reassigned onto PA1)	TIM5_CH2, PWM Generation	PA1
RobotDyn ZC (zero-cross in)	GPIO EXTI, falling edge	PA10
RobotDyn PSM (TRIAC gate out)	GPIO Output (driven by TIM ISR)	PA6
Phase-delay timing	TIM2, one-pulse mode	internal
HAL timebase (RTOS-safe)	TIM6	internal — frees SysTick for FreeRTOS
Debug / telemetry	USART2	PA2 (TX), PA3 (RX) — ST-LINK VCP
Status LED (BSP-owned)	GPIO Output	PA5 (onboard LED, do not reassign)

RobotDyn logic side accepts 3.3V or 5V — connects directly to the Nucleo's 3.3V logic with no level shifting needed. Only the logic-side GND should be common with the STM32; never connect the mains-side of the module to STM32 ground or any other circuitry.

PA5 is owned by the board's BSP (onboard LD2 LED) — it is not reassigned in this design. If a status LED is needed, use a spare pin such as PBO or PC0 instead.

5. STM32CubeMX Configuration

5.1 Clock Configuration

- HSE from ST-LINK, 8 MHz → PLL → SYSCLK = 180 MHz
- APB1 = 45 MHz, APB2 = 90 MHz (standard Nucleo-F446RE max configuration)

5.2 I2C1 — SCD30 (dedicated, slow bus)

- Mode: I2C

- Pins: remapped to PB6 (SCL) and PB7 (SDA) instead of I2C1's default PB8/PB9 — on the chip diagram, click PB6 and select I2C1_SCL, then click PB7 and select I2C1_SDA
- Speed: Standard Mode, 100 kHz (SCD30 maximum per datasheet)
- Increase HAL I2C timeout to accommodate clock-stretch up to ~150 ms
- NVIC: enable I2C1 event + error interrupts if using IT/DMA mode (recommended)

5.3 I2C2 — SPS30 + BME680

- Mode: I2C
- Speed: 100 kHz (SPS30 constraint)
- Standard or DMA mode is fine

5.4 ADC1 — MQ-7 (sensing)

- Channel IN0 (PA0), 12-bit resolution
- Single channel, software or timer-triggered
- F446 has no hardware oversampler — average ~16 samples in software

5.5 TIM5 — MQ-7 Heater Cycle (PWM on PA1)

The MQ-7 heater is driven by PWM on PA1 using TIM5 Channel 2, rather than a plain GPIO on/off switch. PA1 was originally reserved as a plain GPIO output for this purpose; it is now reconfigured as TIM5_CH2 instead, since PA1 supports this alternate function directly on the F446 — no new pin is introduced. TIM5 (not TIM4, and deliberately not TIM2) is used so the heater PWM has its own independent counter/prescaler/ARR, completely decoupled from TIM2's dimmer phase-delay timing — sharing TIM2 between the heater PWM and the AC dimmer would couple two unrelated timing domains onto the same period register, which is avoided here.

- Clock Source: Internal Clock
- Channel 2: PWM Generation CH2 (assign PA1 = TIM5_CH2 on the chip diagram — click PA1, select TIM5_CH2 instead of GPIO_Output)
- Prescaler: sized for a 1 ms tick (e.g. PSC = 89999 if APB1 timer clock = 90 MHz)
- Counter Period (ARR): 149999 → 150 s total period, matching the classic MQ-7 datasheet 60 s-high / 90 s-low heater cycle
- Pulse (CCR2): 60000 → 60,000 ms high time out of the 150 s period (adjust ratio to match the exact duty cycle target)
- Channel Polarity: High (non-inverted)
- TIM5 update interrupt (optional): enable only if an ADC read must be triggered precisely at a fixed point in the heater cycle; not required for basic operation

⚠ Plain PWM only switches a single voltage level on/off — it cannot itself produce the MQ-7's two-level heater drive (e.g. 5V high-power phase / 1.4V low-power phase) if the board's design requires the actual analog voltage to change. If the breakout board fixes the heater at a constant 5V and skips the 1.4V phase (common on many ready-made MQ-7 modules), this PWM on/off timing is sufficient. If true two-voltage drive is required, the PWM signal should instead switch a MOSFET that selects between two supply rails, or an additional analog switching stage is needed on the hardware side.

PA1 was reassigned from plain GPIO_Output to TIM5_CH2 in CubeMX — in the pin popup on the chip diagram, this replaces the earlier GPIO_Output selection rather than adding a second pin. No separate heater-control pin exists any more; PA1 is the sole heater drive line.

5.6 GPIO

- PA6 — Output, push-pull, low speed (RobotDyn PSM / gate drive, toggled from TIM2 ISR — not configured as a PWM channel)
- PA10 — GPIO_EXTI10, falling edge, pull-up enabled (RobotDyn ZC input — confirm actual polarity on a scope)
- PA5 — left untouched (BSP-owned onboard LED, not reassigned)

5.7 EXTI — Zero-Cross Input

- Configure the ZC pin's EXTI line, enable NVIC interrupt
- HAL_GPIO_EXTI_Callback() computes the phase delay and arms TIM2 one-pulse mode
- Priority: see Section 5.10 — FreeRTOS-aware NVIC scheme

5.8 TIM2 — Phase Delay (One-Pulse Mode)

- Mode: One Pulse Mode
- Prescaler: set for ~10 μ s/tick from the actual APB1 Timer Clock shown in the Clock Configuration tab (e.g. PSC = 899 if that clock is 90 MHz, giving a 100 kHz / 10 μ s tick)
- ARR: computed at runtime from desired power % (0 to ~90-95% of the 10 ms half-cycle window)
- Enable Update interrupt — fires the PSM gate pulse once, then auto-stops

5.9 USART2 — Debug / Telemetry

- 115200 baud, 8N1, via ST-LINK VCP (PA2/PA3) for logging sensor values during tuning

5.10 RTOS Configuration — FreeRTOS Retained

The project uses FreeRTOS (CMSIS_V2 interface). Two RTOS-specific settings must be addressed correctly, otherwise timing and interrupt safety break silently at runtime.

5.10.1 HAL Timebase Source — move off SysTick

FreeRTOS takes over SysTick for its own scheduler tick. If HAL's millisecond timebase (HAL_GetTick(), HAL_Delay()) also uses SysTick, the two compete for the same interrupt, corrupting HAL timing once the scheduler is running. CubeMX itself warns about this when FreeRTOS is enabled.

- System Core → SYS → Timebase Source: change from SysTick to a free general-purpose timer
- Selected: TIM6 (unused elsewhere in this design — do not pick TIM2, which is reserved for the dimmer phase delay, or TIM5, reserved for the MQ-7 heater PWM)
- No manual parameter tuning needed — CubeMX configures TIM6 automatically for a 1 ms HAL tick

5.10.2 NVIC Priority Scheme — FreeRTOS-safe

FreeRTOS enforces a syscall priority threshold (configMAX_SYSCALL_INTERRUPT_PRIORITY, commonly priority 5 with 4 preemption bits / 0 subpriority bits configured). Interrupts numerically below this threshold must never call any FreeRTOS API, even *FromISR variants — they must remain pure register-level code. CubeMX's NVIC dropdown enforces this by restricting selectable priorities once FreeRTOS is enabled, so the original bare-metal scheme (priorities 0/1 for EXTI/TIM2) is not selectable and does not apply here.

- Priority Group: 4 bits for pre-emption priority, 0 bits for subpriority
- EXTI15_10 (zero-cross) and TIM2 (gate fire) are kept dumb (pure GPIO/timer register writes, no FreeRTOS calls) and given the highest priority available within the RTOS-permitted range

- I2C1 / I2C2 / USART2 ISRs are allowed to call FreeRTOS *FromISR functions (e.g. to hand sensor data to a queue) since they sit safely within the permitted band

Interrupt Source	Preemption Priority	Rationale
EXTI15_10 (zero-cross)	5	Highest priority available within the FreeRTOS-safe range; must never be delayed — timing anchor for all TRIAC firing; no FreeRTOS API calls
TIM2 global (gate pulse timing)	6	Must fire precisely relative to the zero-cross event; no FreeRTOS API calls
I2C1 event/error (SCD30)	8	May safely call FreeRTOS *FromISR functions
I2C2 event/error (SPS30 + BME680)	8	May safely call FreeRTOS *FromISR functions
USART2 global	10	May safely call FreeRTOS *FromISR functions
FreeRTOS kernel (SysTick, PendSV, SVC)	15 (auto-managed)	Managed by the RTOS — do not override

Verify the actual configMAX_SYSCALL_INTERRUPT_PRIORITY value under Middleware → FREERTOS → Config parameters rather than assuming 5 — if it differs, shift the EXTI/TIM2/I2C/USART priorities accordingly while preserving the same ordering (dimmer ISRs at the lowest permitted numeric value, sensor/comms ISRs safely above it).

A few microseconds of additional worst-case jitter from priority 5 vs. priority 0 is negligible against a 10 ms (50 Hz) or 8.33 ms (60 Hz) mains half-cycle window, so this adjustment carries no meaningful cost to dimmer timing accuracy.

Because tasks now handle sensor polling instead of a single superloop, structure the application as separate FreeRTOS tasks (e.g. vTaskSCD30, vTaskSPS30_BME680, vTaskMQ7, vTaskFanControl) communicating sensor readings and the computed fan-power value via queues, rather than shared volatile globals.

6. Software Architecture

```
main.c
├── Init all HAL peripherals (CubeMX generated)
├── MX_FREERTOS_Init() – creates tasks and queues
├── vTaskSCD30           – I2C1 read every ~2s, pushes reading to xQueueSensorData
├── vTaskSPS30_BME680    – I2C2 reads (SPS30 + BME680), pushes to xQueueSensorData
├── vTaskMQ7             – reads ADC1 (PA0); heater cycle runs autonomously via TIM5 PWM
├── (PA1)
├── vTaskFanControl       – consumes xQueueSensorData, computes fan-power %,
│                           writes result for the dimmer ISR to read
├── vTaskTelemetry        – logs sensor + fan state via USART2
├── ISR: EXTI zero-cross  → arm TIM2 one-pulse mode (pure register writes, no RTOS calls)
├── ISR: TIM2 update      → fire TRIAC gate / PSM pulse (pure GPIO write, no RTOS calls)
├── TIM5 PWM (hardware)   → autonomous MQ-7 heater duty cycle on PA1, no CPU/ISR involvement
```

Given the mix of I2C/analog sensors with differing timing needs (SCD30 measurement cadence, MQ-7 heater cycling, BME680 gas-heating cycles) alongside a hard real-time TRIAC-firing requirement, sensor acquisition is organized as separate FreeRTOS tasks communicating through queues. The zero-cross and TIM2 ISRs remain deliberately "dumb" — pure register-level GPIO/timer manipulation only, with no FreeRTOS API calls — so they can safely run at a priority level the RTOS syscall-safety rule would otherwise forbid from touching kernel objects. See Section 5.10.2 for the exact priority split.

7. Reference Dimmer Driver Code (HAL)

The following illustrates the zero-cross → phase-delay → gate-fire logic. For production use, replace the busy-wait pulse width with a second timer (e.g. TIM3) in one-pulse mode to avoid any blocking inside an ISR.

```
// Global state
volatile uint32_t phase_delay_ticks = 0; // 0 (full power) .. MAX (min power)
volatile uint8_t fan_power_percent = 0; // 0-100, set by control algorithm

#define MAINS_HALF_CYCLE_US 10000 // 50Hz -> 10ms; use 8333 for 60Hz
#define TIM2_TICK_US 10 // PSC configured so 1 tick = 10us
#define MAX_DELAY_TICKS ((MAINS_HALF_CYCLE_US / TIM2_TICK_US) - SAFETY_MARGIN)

// EXTI callback - fires on every zero cross
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_10)
    {
        phase_delay_ticks = MAX_DELAY_TICKS -
            ((fan_power_percent * MAX_DELAY_TICKS) / 100);

        __HAL_TIM_SET_AUTORELOAD(&htim2, phase_delay_ticks);
        __HAL_TIM_SET_COUNTER(&htim2, 0);
        HAL_TIM_Base_Start_IT(&htim2); // one-pulse mode: auto-stops after firing
    }
}

// TIM2 update callback - fires once, at the calculated delay after ZC
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET); // fire PSM
        // ~50-100us pulse width - use a second one-pulse timer in production
        for (volatile int i = 0; i < 400; i++) __NOP();
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
    }
}
```

fan_power_percent is written by *vTaskFanControl* and only read by the EXTI ISR. A plain volatile shared variable is sufficient here — reading a value that is one control-loop iteration stale for a single 10 ms half-cycle is harmless for a slowly-varying fan-speed signal, and no queue or mutex is needed for this particular value.

The MQ-7 heater cycle requires no software logic at all once TIM5 is configured in PWM mode: the 60s-high/90s-low (or equivalent) duty cycle runs autonomously in hardware on PA1. *vTaskMQ7* only needs to read ADC1 at the appropriate point in the cycle.

8. Suggested Next Steps

- Verify actual RobotDyn ZC signal polarity and idle state with an oscilloscope before finalizing EXTI edge configuration.
- Implement the non-blocking gate-pulse version using a second timer (e.g. TIM3) for the ~50-100 μs pulse width, replacing the busy-wait loop shown in Section 7.
- Confirm the actual configMAX_SYSCALL_INTERRUPT_PRIORITY value in the generated FreeRTOSConfig.h and adjust the NVIC table in Section 5.10.2 if it differs from the assumed value of 5.
- Define the FreeRTOS queue(s) and task structure (*vTaskSCD30*, *vTaskSPS30_BME680*, *vTaskMQ7*, *vTaskFanControl*, *vTaskTelemetry*) and the sensor-data struct passed between them.

- Develop sensor driver skeletons for SPS30, BME680, SCD30, and MQ-7 under HAL, starting with SCD30 given its clock-stretching behavior.
- Confirm whether the specific MQ-7 breakout board drives the heater at a constant voltage or requires the two-level (5V/1.4V) drive scheme, to determine if the TIM5 PWM timing alone is sufficient or additional analog switching hardware is needed.
- Characterize the target fan motor's usable dimming range (phase-cut vs. cycle-skipping) before finalizing the control algorithm.